

Отчёт по лабораторной работе №2

Основные модели

true

Цель работы

Цель данной лабораторной работы — исследовать основные математические модели: модель распространения инфекционного заболевания (SIR) и модель динамики популяций хищник-жертва (Лотки-Вольтерры).

Задание

— Создать рабочий каталог для лабораторной работы. — Установить необходимые пакеты. — Выполнить предложенный код. — Преобразовать код в литературный стиль. — Добавить вычисления для набора параметров. — Сгенерировать код, notebook и документацию для варианта с параметрами. — Выполнить расчёты с параметрами. — Сгенерировать из литературного кода: — чистый код; — jupyter notebook; — документацию в формате Quarto. — Выполнить код из jupyter notebook. — Интегрировать документацию в формате Quarto в отчёт.

Теоретическое введение

Модель SIR

Модель SIR — классическая математическая модель эпидемиологии, описывающая динамику распространения инфекционного заболевания в закрытой популяции. Модель делит всю популяцию на три компартмента:

- S (Susceptible, Восприимчивые) — особи, не имеющие иммунитета и способные заразиться при контакте с инфицированными.
- I (Infectious, Инфицированные) — особи, болеющие в данный момент и способные передавать инфекцию.

— R (Recovered, Выздоровевшие) — особи, переболевшие и приобретшие иммунитет.

Система дифференциальных уравнений модели:

$$\frac{dS}{dt} = -\beta c \frac{I}{N} S, \quad \frac{dI}{dt} = \beta c \frac{I}{N} S - \gamma I, \quad \frac{dR}{dt} = \gamma I$$

где β — вероятность заражения при контакте, c — среднее число контактов, γ — скорость выздоровления $N = S + I + R$ — общая численность популяции. Ключевым параметром модели является базовое репродуктивное число $R_0 = c\beta/\gamma$, определяющее характер распространения болезни: при $R_0 > 1$ возникает эпидемия, при $R_0 \leq 1$ — нет.

Модель Лотки-Вольтерры

Модель Лотки-Вольтерры описывает динамику взаимодействия двух популяций — жертв и хищников. Была независимо предложена Альфредом Лоткой (1925) и Витторио Вольтеррой (1926). Система уравнений:

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad \frac{dy}{dt} = \delta xy - \gamma y$$

где x — численность жертв, y — численность хищников, α — скорость естественного прироста жертв, β — коэффициент поедания жертв хищниками, δ — коэффициент прироста хищников за счёт поедания жертв, γ — естественная смертность хищников. Система имеет стационарную точку $x^* = \gamma/\delta$, $y^* = \alpha/\beta$, вокруг которой решения образуют замкнутые орбиты — популяции колеблются периодически.

Выполнение лабораторной работы

Подготовка рабочего каталога

— Перейдём из каталога первой лабораторной в общий каталог курса и создадим директорию `lab02`. Внутри неё создадим поддиректории `presentation` и `report`, а также `project` — рабочее пространство DrWatson ([рис. @fig-001]).

— Перейдём в каталог `project`, запустим Julia и инициализируем новый проект DrWatson командой `initialize_project(".").`


```

julia> using Pkg
julia> Pkg.add("SimpleDiffEq")
Resolving package versions...
Installing Parameters v0.12.0
Installing SimpleDiffEq v1.12.0
Installing StaticArrays v1.9.10
Updating "/julia/environments/v1.12/Project.toml"
[80bca320] + SimpleDiffEq v1.12.0
Updating "/julia/environments/v1.12/Manifest.toml"
# [0b0d13e] + Parameters v0.12.0
# [80bca320] + SimpleDiffEq v1.12.0
# [90177ff4] + StaticArrays v1.9.10
Info Packages marked with * have new versions available but compatibility constraints restrict them from upgrading. To see why use 'status --outdated -m'
Precompiling packages finished.
11 dependencies successfully precompiled in 32 seconds. 345 already precompiled.
julia> Pkg.add("BenchmarkTools")
Resolving package versions...
Updating "/julia/environments/v1.12/Project.toml"
[6e9a2f72] + BenchmarkTools v0.5.0
Updating "/julia/environments/v1.12/Manifest.toml"
# [6e9a2f72] + BenchmarkTools v0.5.0
# [3bb9d95c] + Profile v1.1.0
Precompiling packages finished.
1 dependency successfully precompiled in 15 seconds. 357 already precompiled.
julia> Pkg.add("LaTeXStrings")
Resolving package versions...
Updating "/julia/environments/v1.12/Project.toml"
[08e9077c] + LaTeXStrings v1.4.0
Updating "/julia/environments/v1.12/Manifest.toml"

```

Figure 3: Установка пакетов SimpleDiffEq, BenchmarkTools, LaTeXStrings

```

julia> Pkg.add("StatsPlots")
Resolving package versions...
Installing Random123 v1.3.1
Installing HypergeometricFunctions v0.3.20
Installing Distributions v0.25.11
Installing Distances v0.10.12
Installing OffsetArrays v1.17.0
Installing StatsFuns v2.1.0
Installing Wavelets.jl v0.4.2
Installing Clustering v0.12.0
Installing Ratios v0.4.5
Installing AbstractFFTs v1.5.0
Installing AbstractTrees v0.4.5
Installing StatsPlots v1.5.0
Installing KernelDensity v0.6.12
Installing ChainRulesCore v1.20.1
Installing Interpolations v0.16.2
Installing MultivariateStats v0.10.0
Installing Arpack.jl v1.5.2+0
Installing eDagTy v0.6.7
Installing IntegerMathUtils v0.1.1
Installing Reus v0.9.0
Installing Primes v0.5.7
Installing TableOperations v1.2.0
Installing FFTs v0.3.1
Installing Arpack v0.5.4
Installing FilterRules v1.16.0
Installing DeltaAlgorithms v1.1.0
Installing Observables v0.5.5
Installing WoodburyMatrices v1.1.0
Installing Distributions v0.25.11
Installing QuadGK v1.1.3
Installing Plots v1.12.0
Updating "/julia/environments/v1.12/Project.toml"
[7a297272] + StatsPlots v1.5.0
Updating "/julia/environments/v1.12/Manifest.toml"
# [7a297272] + StatsPlots v1.5.0
# [21216a1c] + AbstractFFTs v1.5.0
# [21216a1c] + AbstractTrees v0.4.5
# [70f322c2] + Arpack v0.5.4
# [18722f3f] + DeltaAlgorithms v1.1.0
# [2308220c] + ChainRulesCore v1.20.1
# [3bb9d95c] + Profile v1.1.0

```

Figure 4: Установка пакета StatsPlots

```

julia> cd("../scripts")
julia> touch sir_ode.jl
julia> ls
sir_ode.jl

```

Figure 5: Создание скрипта sir_ode.jl

пакетов (DifferentialEquations, SimpleDiffEq, Tables, DataFrames, StatsPlots, LaTeXStrings, Plots, BenchmarkTools), определение функции `sir_ode!` с уравнениями модели, задание параметров ($\beta = 0.05$, $c = 10.0$, $\gamma = 0.25$, начальные условия $S_0 = 990$, $I_0 = 10$, $R_0 = 0$), расчёт базового репродуктивного числа и численное решение через `ODEProblem` ([рис. @fig-004]).

```

1 using DifferentialEquations
2 using SimpleDiffEq
3 using Tables
4 using DataFrames
5 using StatsPlots
6 using LaTeXStrings # Для красивого отображения формул на графиках
7 using Plots
8 using BenchmarkTools
9
10 script_name = splitext(basename(PROGRAM_FILE))[1]
11 mkpath(joinpath(script_name, "plots"))
12 mkpath(joinpath(script_name, "data"))
13
14 function sir_ode!(du, u, p, t)
15     (S, I, R) = u
16     (beta, c, gamma) = p
17     N = S + I + R
18     @inbounds begin
19         du[1] = beta * c * I / N * S
20         du[2] = -beta * c * I / N * S - gamma * I
21         du[3] = gamma * I
22     end
23     nothing
24 end
25
26 # Параметры модели
27 dt = 0.1
28 tmax = 40.0
29 tspan = (0.0, tmax)
30 u0 = [990.0, 10.0, 0.0] # S, I, R
31 p = [0.05, 10.0, 0.25] # beta, c, gamma
32
33 # Расчет базового репродуктивного числа
34 R0 = (p[2] * p[1]) / p[3] # R0 = (c * beta) / gamma
35
36 # Создание и решение задачи
37 prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
38 sol_ode = solve(prob_ode, dt = dt)
39
40 # Подготовка данных в DataFrame
41 df_ode = DataFrame{Tables.Table{sol_ode}}
42 rename!(df_ode, ["S", "I", "R"])
43 df_ode[1, :t] = sol_ode.t
44 df_ode[1, :N] = df_ode.S + df_ode.I + df_ode.R # Общая численность популяции
45
46 # Вывод параметров модели
47 println("Параметры модели: S, I, R")
48 println("R0 (критический порог) = ", R0)
49
50 # Вывод графиков
51

```

Figure 6: Код скрипта `sir_ode.jl` в VS Code

Запуск и результаты

— Запустим скрипт командой `julia --project=. scripts/sir_ode.jl`. Программа выводит параметры модели, результаты анализа: пиковое число заражённых $I_{max} = 154.8$, время достижения пика $t_{peak} = 19.1$ дней, итоговое число переболевших $R(\infty) = 775.7$ (доля 77.6%), а также теоретический порог коллективного иммунитета 50.0% при $R_0 = 2.0$ ([рис. @fig-007]).

— Проверим, что скрипт сохранил все графики. Команда `find plots -type f` перечисляет семь файлов: `sir_phase_portrait.png`, `sir_log_scale.png`, `sir_percentages.png`, `sir_infected.png`, `sir_effective_R.png`, `sir_main.png`, `sir_panel.png` ([рис. @fig-008]).

```

julia> exit()
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ julia --project=. scripts/sir_ode.jl
Параметры модели SIR:
β (вероятность заражения) = 0.85
c (среднее число контактов) = 18.0
γ (скорость выздоровления) = 0.25
R0 = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 = 0.0
Could not create decoration from factory: Running with no decorations.

Бенчмарк решения:
тип: ANNA3 РЕЗУЛЬТАТОБ ана
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_пик = 154.0
Время достижения пика: t_пик = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.0%

Теоретический анализ:
- Порог коллективного иммунитета: 50.0%
- Теоретический пик при S/N = 1/R0 = 0.5

```

Figure 7: Вывод результатов модели SIR

```

- Теоретический пик при S/N = 1/R0 = 0.5
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ find plots -type f
plots/sir_ode/sir_phase_portrait.png
plots/sir_ode/sir_log_scale.png
plots/sir_ode/sir_percentages.png
plots/sir_ode/sir_infected.png
plots/sir_ode/sir_effective_R.png
plots/sir_ode/sir_main.png
plots/sir_ode/sir_panel.png
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ find data -type f
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ find notebooks -type f

```

Figure 8: Список сгенерированных графиков модели SIR

Подготовка Quarto-документации

— Создадим каталог `markdown/sir_ode` для Quarto-документации модели SIR и `markdown/lv_ode` для модели Лотки-Вольтерры. Структура каталога `markdown` и содержимое `notebooks` подтверждают корректность (рис. @fig-015)).

```

~/2026-1--study--simulation-modeling/labs/lab02/project$ mkdir -p markdown/sir_ode
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ mkdir -p markdown/lv_ode
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ find markdown -maxdepth 2 -type d
markdown
markdown/sir_ode
markdown/lv_ode
@javadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ ls notebooks/
djavadave@djavadave-simodeling:~/2026-1--study--simulation-modeling/labs/lab02/project$ ls -R markdown
markdown:
lv_ode  sir_ode
markdown/lv_ode:
markdown/sir_ode:

```

Figure 9: Создание каталогов `markdown/sir_ode` и `markdown/lv_ode`

— В редакторе VS Code откроем файл `markdown/sir_ode/sir_ode.qmd`. Документ содержит заголовок «Модель SIR», разделы с блоками Julia-кода, активацию проекта, подключение всех необходимых пакетов (рис. @fig-020)).

— Перейдём в каталог `markdown/sir_ode` и запустим `quarto render sir_ode.qmd`. Quarto последовательно выполняет все 27 блоков Julia-кода: определение функции, задание параметров, численное решение, построение графиков и вывод аналитического анализа (рис. @fig-016)).

— Файловый менеджер подтверждает наличие файла `sir_ode.qmd`

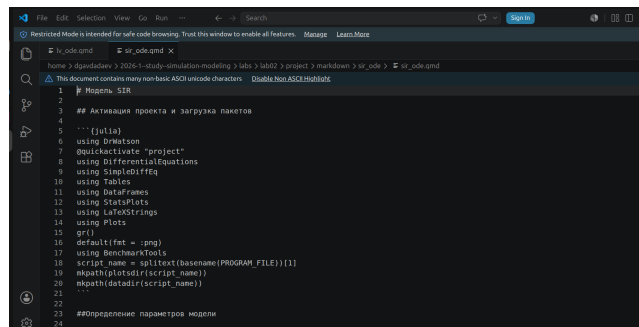


Figure 10: Файл sir_ode.qmd в VS Code

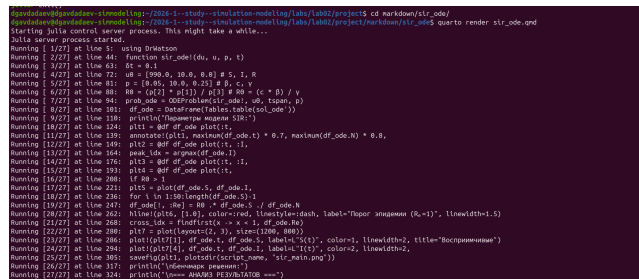


Figure 11: Рендеринг документации sir_ode.qmd через Quarto

в каталоге `markdown/sir_ode` вместе с файлами проекта ([рис. @fig-017]).

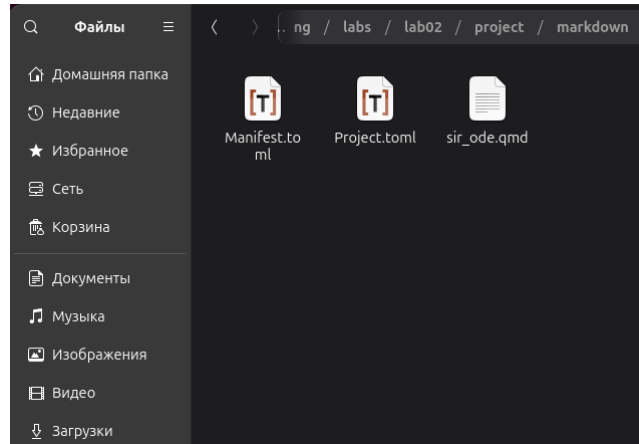


Figure 12: Каталог `markdown/sir_ode` в файловом менеджере

Модель Лотки-Вольтерры

Реализация скрипта

— Создадим файл `scripts/lv_ode.jl`. Список скриптов теперь содержит три файла: `intro.jl`, `lv_ode.jl`, `sir_ode.jl` ([рис. @fig-010]).

```
spavd@spavd@spavd:~/simulation-modeling/labs/lab02/project$ code scripts/lv_ode.jl
spavd@spavd@spavd:~/simulation-modeling/labs/lab02/project$ ls scripts/
intro.jl  lv_ode.jl  sir_ode.jl
spavd@spavd@spavd:~/simulation-modeling/labs/lab02/project$
```

Figure 13: Создание скрипта `lv_ode.jl`

— В редакторе VS Code открыт файл `scripts/lv_ode.jl`. Скрипт подключает пакеты (`DifferentialEquations`, `DataFrames`, `StatsPlots`, `LaTeXStrings`, `Plots`, `Statistics`, `FFTW`), описывает систему уравнений Лотки-Вольтерры с параметрами, определяет функцию `lotka_volterra!` ([рис. @fig-011]).

— Установим пакет `FFTW` (быстрое преобразование Фурье), необходимый для спектрального анализа колебаний популяций ([рис. @fig-012]).

Запуск и результаты

— Запустим скрипт `julia --project=. scripts/lv_ode.jl`. Программа выводит параметры модели ($\alpha = 0.1$, $\beta = 0.02$,

$\delta = 0.01$, $\gamma = 0.3$), начальные условия (жертвы $x_0 = 40.0$, хищники $y_0 = 9.0$), стационарные точки ($x^* = 30.0$, $y^* = 5.0$), доминирующую частоту колебаний жертв 0.2499 Гц и период 4.0 единицы времени ([рис. @fig-013]).

```

in expression starting at /home/dgavdadaev/2026-1--study--simulation-modeling/labs/lab02/project/scripts/lv_ode.jl:225
dgavdadaev@dgavdadaev:~/2026-1--study--simulation-modeling/labs/lab02/project$ julia --project= scripts/lv_ode.jl
Warning: Verbosity toggle: dense_output_save
Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two behaviors.
@ OrdinaryDiffEqCore ./julia/packages/OrdinaryDiffEqCore/WM1eJ/src/solve.jl:1171

Модель Лотки-Вольтерры (хищник-жертва)
=====

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0

Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени

=====
Анализ результатов
=====

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.0, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
Could not create decoration from factory! Running with no decorations.
Warning: No strict ticks found
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:194
Warning: No strict ticks found
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:194
Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:185
Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:185
Warning: No strict ticks found
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:194
Warning: No strict ticks found
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:194
Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:185
Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xscale
@ Plots.jl ./julia/packages/Plots/0080C/src/ticks.jl:185

=====
Анализ чувствительности
=====

```

Figure 16: Вывод результатов модели Лотки-Вольтерры

— После успешного завершения моделирования проверим сохранённые файлы. `find plots -type f` показывает полный набор графиков обеих моделей: 7 файлов SIR и 6 файлов LV (`lv_panel.png`, `lv_phase_portrait.png`, `lv_derivatives.png`, `lv_relative_changes.png`, `lv_dynamics.png`, `lv_spectrum.png`). Команда `find . -maxdepth 2 -type d` демонстрирует полную структуру проекта ([рис. @fig-014]).

Подготовка Quarto-документации

— В редакторе VS Code откроем файл `markdown/lv_ode/lv_ode.qmd`. Документ оформлен в литературном стиле: заголовок «Модель Лотки-Вольтерры», блок активации проекта и загрузки пакетов, блок описания модели с уравнениями ([рис. @fig-019]).

— Файловый менеджер показывает каталог `markdown/lv_ode` с файлом `lv_ode.qmd` ([рис. @fig-018]).

```
=====
Резерпирование завершено успешно!
@spvdsdave@spvdsdave:~/simulation-modeling$ find plots -type f
plots/sir_ode/sir_phase_portrait.png
plots/sir_ode/sir_log_scale.png
plots/sir_ode/sir_percentages.png
plots/sir_ode/sir_infected.png
plots/sir_ode/sir_effective_R.png
plots/sir_ode/sir_main.png
plots/sir_ode/sir_panel.png
plots/lv_ode/lv_panel.png
plots/lv_ode/lv_phase_portrait.png
plots/lv_ode/lv_derivatives.png
plots/lv_ode/lv_relative_changes.png
plots/lv_ode/lv_dynamics.png
plots/lv_ode/lv_spectrum.png
@spvdsdave@spvdsdave:~/simulation-modeling$ find data -type f
@spvdsdave@spvdsdave:~/simulation-modeling$ find . -maxdepth 2 -type d
.
./src
./scripts
./github
./github/workflows
./data
./data/sir_ode
./data/exp_raw
./data/exp_pro
./data/lv_ode
./research
./notebooks
./plots
./plots/sir_ode
./plots/lv_ode
./papers
./git
./git/logs
./git/objects
./git/info
./git/refs
./git/hooks
./test
@spvdsdave@spvdsdave:~/simulation-modeling$
```

Figure 17: Список всех сгенерированных графиков и структура проекта

```
lv_ode.qmd
1 # Модель Лотки-Вольтерры
2
3 ## Активация проекта и загрузка пакетов
4
5 ```[julia]
6 using DrWatson
7 @quickactivate "project"
8 using DifferentialEquations
9 using DataFrames
10 using StatPlots
11 using LaTeXStrings
12 using Plots
13 using Statistics
14 using FFTW
15 script_name = splittest(basename(PROGRAM_FILE))[1]
16 mkpath(plotdir(script_name))
17 mkpath(data_dir(script_name))
18 ...
19
20 ## Описание модели Лотки-Вольтерры
21
22 ```[julia]
23 ...
24 Модель Лотки-Вольтерры (хищник-жертва)
25 Система уравнений:
```

Figure 18: Файл lv_ode.qmd в VS Code

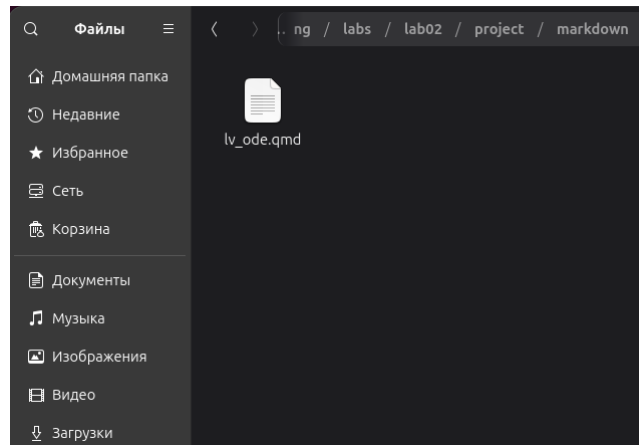


Figure 19: Каталог markdown/lv_ode в файловом менеджере

— Перейдём в каталог markdown/lv_ode и запустим `quarto render lv_ode.qmd`. Quarto последовательно выполняет все 28 блоков Julia-кода, включая спектральный анализ и построение шести графиков ([рис. @fig-021]).

```

Running [ 1/28] at line 5: using Ormatson
Running [ 2/28] at line 22: "
Running [ 3/28] at line 56: p_lv = [0.1, 0] # a: скорость размножения жертв
Running [ 4/28] at line 65: u0_lv = [0.0, 0.0] # начальная популяция
Running [ 5/28] at line 71: tspan_lv = (0.0, 200.0) # длительность симуляции
Running [ 6/28] at line 78: prob_lv = ODEProblem{Lotka_Volterra, u0_lv, tspan_lv, p_lv}
Running [ 7/28] at line 82: df_lv = ODEArray{Float64, 2, 1}
Running [ 8/28] at line 101: df_lv[i, :dprey_dt] = p_lv[i] * df_lv[prey] - p_lv[i] * df_lv[predator]
Running [ 9/28] at line 108: println("=400")
Running [10/28] at line 124: x_star = p_lv[3] / p_lv[1] # стационарная точка для жертв
Running [11/28] at line 135: plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
Running [12/28] at line 150: hline([x_star], :g, :dashed, linestyle=:dash, alpha=0.5, label="x* (равновесие жертв)")
Running [13/28] at line 157: plt2 = plot(df_lv.prey, df_lv.predator,
Running [14/28] at line 172: step = 50 # шаг для отрисовки стрелок
Running [15/28] at line 183: scatter([x_star], [y_star],
Running [16/28] at line 190: x_range = LinRange(0, maximum(df_lv.prey)+1, 100)
Running [17/28] at line 203: plt3 = plot(df_lv.t, [df_lv.dprey dt df_lv.dpredator dt],
Running [18/28] at line 219: df_lv[i, :prey_pct_change] = df_lv.dprey dt / df_lv.prey * 100
Running [19/28] at line 238: function compute_fft(signal, dt)
Running [20/28] at line 249: freq_prey, spectrum_prey = compute_fft(df_lv.prey ./ mean(df_lv.prey), dt_lv)
Running [21/28] at line 268: if length(spectrum_prey) > 0
Running [22/28] at line 280: plt6 = plot(layouts(3, 2), size=(1200, 900))
Running [23/28] at line 301: println("v0 = ", v0)
Running [24/28] at line 318: function find_first_peak(signal, time)
Running [25/28] at line 342: savefig(plt1, plotof(script_name, "lv_dynamics.png"))
Running [26/28] at line 353: println("v0 = ", v0)
Running [27/28] at line 361: function analyze_parameter_sensitivity(param_index, values, param_name)
Running [28/28] at line 381: if false # включение в true для выполнения анализа

```

Figure 20: Рендеринг документации lv_ode.qmd через Quarto

Выполнение Jupyter-ноутбуков

— Откроем сгенерированный ноутбук `sir_ode.ipynb` в JupyterLab. Ноутбук содержит разделы «Активация проекта и загрузка пакетов» и «Определение параметров модели» с отображением системы дифференциальных уравнений ([рис. @fig-022]).

— Выполним ноутбук. Результаты бенчмарка решателя: медианное время 17.600 мкс, среднее 22.588 мкс, оценка памяти 17.39 KiB при 333 аллокациях. Аналитический вывод

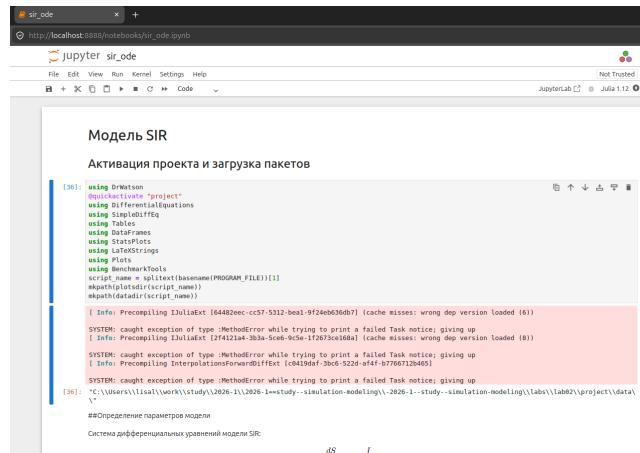


Figure 21: Jupyter-ноутбук sir_ode.ipynb в JupyterLab

совпадает с результатами скрипта: $N = 1000.0$, $I_{max} = 154.8$, $t_{peak} = 19.1$ дней, $R(\infty) = 775.7$ ([рис. @fig-023]).

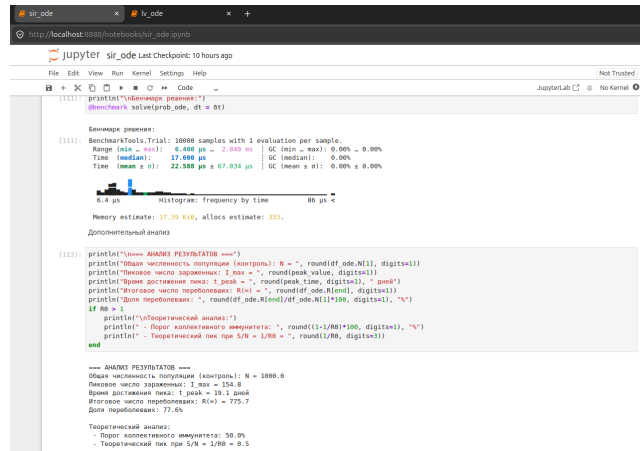


Figure 22: Результаты выполнения ноутбука sir_ode

— Откроем ноутбук lv_ode.ipynb. Ноутбук содержит разделы с построением шести графиков: популяция жертв, хищников, фазовый портрет, скорости изменения, спектр жертв и относительные изменения ([рис. @fig-024]).

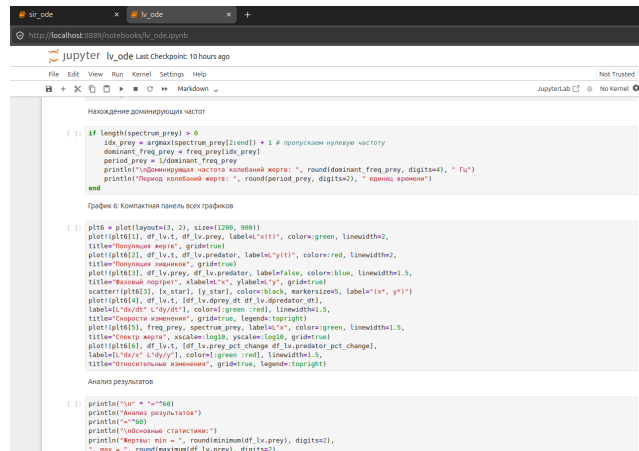


Figure 23: Jupyter-ноутбук lv_ode.ipynb в JupyterLab

Интеграция в отчёт

Сгенерированные Quarto-документы подключаются к итоговому отчёту через директивы `{{< include >}}`.

```
{{< include ../project/markdown/sir_ode/sir_ode.qmd >}}
```

```
{{< include ../project/markdown/lv_ode/lv_ode.qmd >}}
```

Выводы

По итогам лабораторной работы изучены, реализованы и численно исследованы две фундаментальные математические модели. Модель SIR с параметрами $\beta = 0.05$, $c = 10.0$, $\gamma = 0.25$ показала: при $R_0 = 2.0$ эпидемия охватывает 77.6% популяции, пиковое число заражённых достигает 154.8 человек на 19.1-й день. Модель Лотки-Вольтерры с параметрами $\alpha = 0.1$, $\beta = 0.02$, $\delta = 0.01$, $\gamma = 0.3$ при начальных условиях $x_0 = 40$, $y_0 = 9$ продемонстрировала периодические колебания с периодом 4.0 единицы времени вокруг стационарной точки $x^* = 30$, $y^* = 5$. Освоен полный цикл работы: от реализации скриптов Julia до генерации Quarto-документации и выполнения Jupyter-ноутбуков.

Список литературы

1. Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society A. — 1927. — Vol. 115, no. 772. — P. 700-721.
2. Lotka A. J. Elements of Physical Biology. — Baltimore: Williams & Wilkins, 1925.
3. Volterra V. Fluctuations in the Abundance of a Species considered Mathematically // Nature. — 1926. — Vol. 118. — P. 558-560.
4. Имитационное моделирование. Практикум / А. В. Королькова, Д. С. Кулябов. — Москва : РУДН, 2025.